```r
library(e1071)
library(rstan)
library(ggplot2)

#### Pearson Type III Distribution functions ####
# Pearson Type III - PDF
dp3 = function(x, mu, sigma, gamma)
{
  # If skew is sufficiently close to zero, use the Normal distribution.
  if (abs(gamma) < 1E-3)
  {
    return(dnorm(x=x, mean=mu, sd=sigma))
  }
  # Get P3 parameters
  xi = mu-2*sigma/gamma #location
  beta = 0.5*sigma*gamma #scale
  alpha = 4/gamma^2 #shape
  # Set the min and max support for the distribution.
  min = -.Machine$double.xmax
  max = .Machine$double.xmax
  if(beta > 0) {
    min = xi
    max = .Machine$double.xmax
  }
  else{
    min = -.Machine$double.xmax
    max = xi
  }
  # Check if the data is not within the support
  if (x < min || x > max) return(0)
  # Return the PDF
  if (beta > 0)
  {
    return(dgamma(x=x-xi, shape=alpha, scale=abs(beta)))
  }
  else
  {
    return(dgamma(x=xi-x, shape=alpha, scale=abs(beta)))
  }
}

# Pearson Type III - CDF
pp3 = function(x, mu, sigma, gamma)
{
  # If skew is sufficiently close to zero, use the Normal distribution.
  if (abs(gamma) < 1E-3)
  {
    return(pnorm(q=x, mean=mu, sd=sigma))
  }
  # Get P3 parameters
  xi = mu-2*sigma/gamma #location
  beta = 0.5*sigma*gamma #scale
  alpha = 4/gamma^2 #shape
  # Set the min and max support for the distribution.
  min = -.Machine$double.xmax
  max = .Machine$double.xmax
  if(beta > 0) {
    min = xi
    max = .Machine$double.xmax
  }
  else{
    min = -.Machine$double.xmax
    max = xi
  }
  # Check if the data is not within the support
  if (x <= min) return(0)
```

```
  if (x >= max) return(1)
  # Return the CDF
  if (beta > 0)
  {
    return(pgamma(q=x-xi, shape=alpha, scale=abs(beta)))
  }
  else
  {
    return(1-pgamma(q=xi-x, shape=alpha, scale=abs(beta)))
  }
}

# Pearson Type III - Inverse CDF
qp3 = function(p, mu, sigma, gamma)
{
  # If skew is sufficiently close to zero, use the Normal distribution.
  if (abs(gamma) < 1E-3)
  {
    return(qnorm(p=p, mean=mu, sd=sigma))
  }
  # Get P3 parameters
  xi = mu-2*sigma/gamma #location
  beta = 0.5*sigma*gamma #scale
  alpha = 4/gamma^2 #shape
  # Set the min and max support for the distribution.
  min = -.Machine$double.xmax
  max = .Machine$double.xmax
  if(beta > 0) {
    min = xi
    max = .Machine$double.xmax
  }
  else{
    min = -.Machine$double.xmax
    max = xi
  }
  # Check if the data is not within the support
  if (p == 0) return(min)
  if (p == 1) return(max)
  # Return the inverse CDF
  if (beta > 0)
  {
    return(xi + qgamma(p=p, shape=alpha, scale=abs(beta)))
  }
  else
  {
    return(xi - qgamma(p=1-p, shape=alpha, scale=abs(beta)))
  }
}

#### Example 3 Gage Data ####
# https://www.arr-software.org/pdfs/ARR_190514_Book3.pdf
# Hunt River at Singleton Peak Flow
data = c(76.26, 171.87, 218.21, 668.79, 1374.42, 124.12, 276.3, 895.5, 1374.42, 280.18,
202.62, 4052.42, 2323.77, 2536.31, 3315.62, 1232.73, 1391.43, 12525.66, 1099.54,447.75,
478.92, 180.52, 164.36, 229.54, 2125.4, 966.35, 2751.68, 49.03, 76.51, 912.5, 926.67)
# The data needs to be log10 transformed. You can do this up front, or in the R-Stan
function.
# RMC-BestFit uses real-space data and converts to log-space in the code behind.
data = log10(data)

#### Estimate using Bayesian MCMC with R-Stan ####

# This is the code for R-Stan with the Pearson Type III distribution.
# Stan will automatically compile this code to significantly reduce runtimes.
stan_code = 'functions {
                // The log-density for Pearson Type III
```

```
  real p3lpdf(real x, real mu, real sigma, real gamma){

    // If skew is sufficiently close to zero, use the Normal distribution.
    if (fabs(gamma) < 0.001){
      return normal_lpdf(x | mu, sigma);
    }

    // Get P3 parameters
    real xi; // location
    real beta; // scale
    real alpha; // shape

    xi = mu-2.0*sigma/gamma;
    beta = 1/(0.5*sigma*gamma);
    alpha = 4/(gamma*gamma);

    // Set the min and max support for the distribution.
    real xMin;
    real xMax;

    if (beta > 0){
      xMin = xi;
      xMax = 1.7*10^308;
    } else {
      xMin = -1.7*10^308;
      xMax = xi;
    }

    // Check if the data is not within the support
    if (x < xMin || x > xMax){
      // If the x value is not within support, penalize the log-likelihood.
      return -1.7*10^308;
    }

    // Return the log-likelihood
    if (beta > 0){
      return gamma_lpdf(x-xi | alpha, fabs(beta));
    } else {
      return gamma_lpdf(xi-x | alpha, fabs(beta));
    }

  }
}
data {
int N;
real x[N];
}
parameters {
real mu;
real<lower = 0> sigma;
real<lower = -2, upper = 2> gamma;
}
model {
  // data likelihood
  for(n in 1:N){
    target +=  p3lpdf(x[n], mu, sigma, gamma);
  }
  // prior likelihood
  // Here I am using the same defualt flat priors from RMC-BestFit
  target += uniform_lpdf(mu|0,5);
  target += uniform_lpdf(sigma|0,5);
  // Add Jeffreys prior for sigma
  target += -log(sigma);
  target += uniform_lpdf(gamma|-2,2);
}'
```

```r
# Create the Stan model
model = stan_model(model_code=stan_code)

# Set the MCMC simulation options to be closely compatible with RMC-BestFit.
set.seed(12345) # set the seed for repeatability
options(mc.cores = parallel::detectCores())
mcmc = sampling(model, data=list(x=data, N=length(data)), warmup=30000, iter=110000,
chains=4, thin=20)

# Output the summary statistics for the Bayesian estimated parameters.
print(mcmc, probs=c(0.05, 0.5, 0.95))
#         mean se_mean  sd      5%    50%    95% n_eff Rhat
# mu      2.79    0.00 0.11    2.61   2.79   2.97 16281    1
# sigma   0.63    0.00 0.09    0.50   0.61   0.80 15101    1
# gamma   0.12    0.00 0.49   -0.65   0.11   0.95 15785    1
# lp__  -32.70    0.01 1.20 -35.03 -32.39 -31.42 16452    1

# The solution from RMC-BestFit is:
#           mean      sd      5%     50%     95%  Rhat
# mu     2.7947  0.1180  2.6042  2.7925  2.9910     1
# sigma  0.6412  0.1005  0.5051  0.6276  0.8219     1
# gamma  0.1362  0.5072 -0.6630  0.1213  0.9918     1

# Plot Markov Chain trace plots.
stan_trace(mcmc, inc_warmup = FALSE)

# Plot kernel density estimates for each parameter.
stan_dens(mcmc, pars = "mu")
stan_dens(mcmc, pars = "sigma")
stan_dens(mcmc, pars = "gamma")

#### 2. Create the flood frequency curve ####

# Get the last 10,000 posterior parameter sets
output = extract(mcmc)
posterior.mu = output$mu[6001:16000]
posterior.sigma = output$sigma[6001:16000]
posterior.gamma = output$gamma[6001:16000]
posterior.llh = output$lp__[6001:16000]
# get index with maximum likelihood
p.mode.idx = which.max(posterior.llh)

# AEP values for frequency curve plot
AEPs = c( 0.001, 0.002, 0.005, 0.01, 0.02, 0.05, 0.1, 0.2, 0.3, 0.5,0.7, 0.8, 0.9)

# create posterior mode frequency curve
posterior.mode = numeric(length(AEPs))
for (i in 1:length(AEPs)){
  posterior.mode[i] = 10^qp3(1-AEPs[i], posterior.mu[p.mode.idx], posterior.sigma[
  p.mode.idx], posterior.gamma[p.mode.idx])
}

# Define the percentile confidence levels
CIs = c(0.05, 0.95)

Realz = 10000
posterior.Parameters = matrix(nrow = Realz, ncol = 3) # matrix for parameter output
posterior.Quantiles = matrix(nrow = Realz, ncol = length(AEPs)) # matrix for quantile
output
credible.intervals = matrix(nrow = length(CIs), ncol = length(AEPs)) # matrix for CI
output
posterior.predictive = matrix(nrow = 100, ncol = 2) # matrix for posterior predictive
curve output
# keep track of min and max values
maxX = -Inf
minX = Inf
```

```
257
258 # Loop through parameter sets and record parameter and quantile values
259 for (i in 1:Realz) {
260
261   # Record bootstrapped parameters
262   posterior.Parameters[i,] = c(posterior.mu[i], posterior.sigma[i], posterior.gamma[i])
263
264   # Record quantiles
265   for (j in 1:length(AEPs)) {
266     posterior.Quantiles[i,j] = qp3(1-AEPs[j], posterior.mu[i], posterior.sigma[i],
      posterior.gamma[i])
267     # keep track of min and max x values
268     maxX = max(maxX, posterior.Quantiles[i,j])
269     minX = min(minX, posterior.Quantiles[i,j])
270   }
271
272 }
273
274 # Now calculate the credible intervals
275 for (i in 1:length(CIs)) {
276   for (j in 1:length(AEPs)) {
277     credible.intervals[i,j] = 10^quantile(posterior.Quantiles[,j], probs = CIs[i])
278   }
279 }
280
281 # Now calculate the posterior predictive curve
282 # First get X bins. Use log-scale
283 delta = (maxX - minX)/99
284 posterior.predictive[1,1] = minX
285 for (i in 2:100) {
286   posterior.predictive[i,1] = (posterior.predictive[i-1,1] + delta)
287 }
288
289 # Then get mean probabilities
290 for (i in 1:100) {
291   # Loop over all parameter sets and record NEPs
292   pVals = numeric(Realz)
293   for (j in 1:Realz) {
294     pVals[j] = pp3(posterior.predictive[i,1], posterior.Parameters[j,1],
      posterior.Parameters[j,2], posterior.Parameters[j,3])
295   }
296   # Record mean AEP
297   posterior.predictive[i,2] = 1-mean(pVals)
298 }
299
300 # Finally, we need to interpolate the predictive curve given the list of AEPs
301 posterior.predictive = 10^approx(x = posterior.predictive[,2],  y = posterior.predictive[
    ,1], xout = AEPs)$y
302
303 # Create points for observed data
304 # Points are plotted using the Weibull plotting position formula.
305 # You can change these to whatever formula you would like.
306 d = sort(data, decreasing = TRUE)
307 observed = matrix(nrow = length(d), ncol=2)
308 for (i in 1:length(d)){
309   observed[i,1] = (i/(length(d)+1))
310   observed[i,2] = 10^d[i]
311 }
312 observed = as.data.frame(observed)
313 colnames(observed)= c("pp", "flow")
314
315 ### Create Report Quality Plot ####
316 AEPBreaks = c(0.0000001,0.000001, 0.00001, 0.0001, 0.001,  0.01, 0.1, 0.5, 0.9, 0.99)
317 AEPLabels = c("1E-7","1E-6", "1E-5", "1E-4", "0.001",  "0.01", "0.1", "0.5", "0.9",
    "0.99")
318 curves = data.frame(aep = AEPs, p95 = credible.intervals[2,], p05 = credible.intervals[1,
```

```
], pred = posterior.predictive, mode = posterior.mode)
flikeCurves = data.frame(aep = c(0.1, 0.02, 0.01, 0.002), p95 = c(8408, 51010, 107122, 570635), p05 = c(2229,   5502,   7188,   11507),
                         eaep = c(0.101, 0.0232, 0.0136,   0.0048), eQ = c(3929, 12786, 19572, 47034))

ggplot()+
  geom_ribbon(data=curves, aes(x=qnorm(1-aep), y=mode, ymin=p05, ymax=p95), fill = "steelblue", alpha = 0.3)+
  geom_line(data=curves, aes(x=qnorm(1-aep), y=pred), linetype = "dotdash", color="blue", size = 0.5)+
  geom_line(data=curves, aes(x=qnorm(1-aep), y=mode), size = 0.5)+
  geom_line(data=flikeCurves, aes(x=qnorm(1-aep), y=p95), linetype = "dashed", color="red", size = 0.5) +
  geom_line(data=flikeCurves, aes(x=qnorm(1-aep), y=p05), linetype = "dashed", color="red", size = 0.5) +
  geom_line(data=flikeCurves, aes(x=qnorm(1-eaep), y=eQ), linetype = "twodash", color="red", size = 0.5) +
  geom_point(data=flikeCurves, aes(x=qnorm(1-aep), y=p95), color="red", size = 1) +
  geom_point(data=flikeCurves, aes(x=qnorm(1-aep), y=p05), color="red", size = 1) +
  geom_point(data=flikeCurves, aes(x=qnorm(1-eaep), y=eQ), color="red", size = 1) +
  geom_point(data=observed, aes(x=qnorm(1-pp), y=flow), fill = "gray", color = "black", pch=21, size = 2)+
  xlab("Annual Exceedance Probability") +
  ylab("Peak Flow (CMS)") +
  ggtitle("Comparison of R-Stan to FLIKE for Example #3", subtitle = "Jeffreys' Prior for Sigma") +
  scale_y_continuous(limits = c(10, 1E7), trans = "log10", breaks = c(10, 100, 1E3, 1E4, 1E5, 1E6, 1E7), labels = scales::comma) +
  scale_x_continuous(limits = c(qnorm(0.1), qnorm(1-1E-3)), breaks=qnorm(1-AEPBreaks), labels=AEPLabels) +
  theme_classic() +
  theme(axis.text = element_text(size = 10)) +
  theme(panel.grid.major.x = element_line( size=.1, color="gray90"))  +
  theme(panel.grid.major.y = element_line( size=.1, color="gray90"))
```